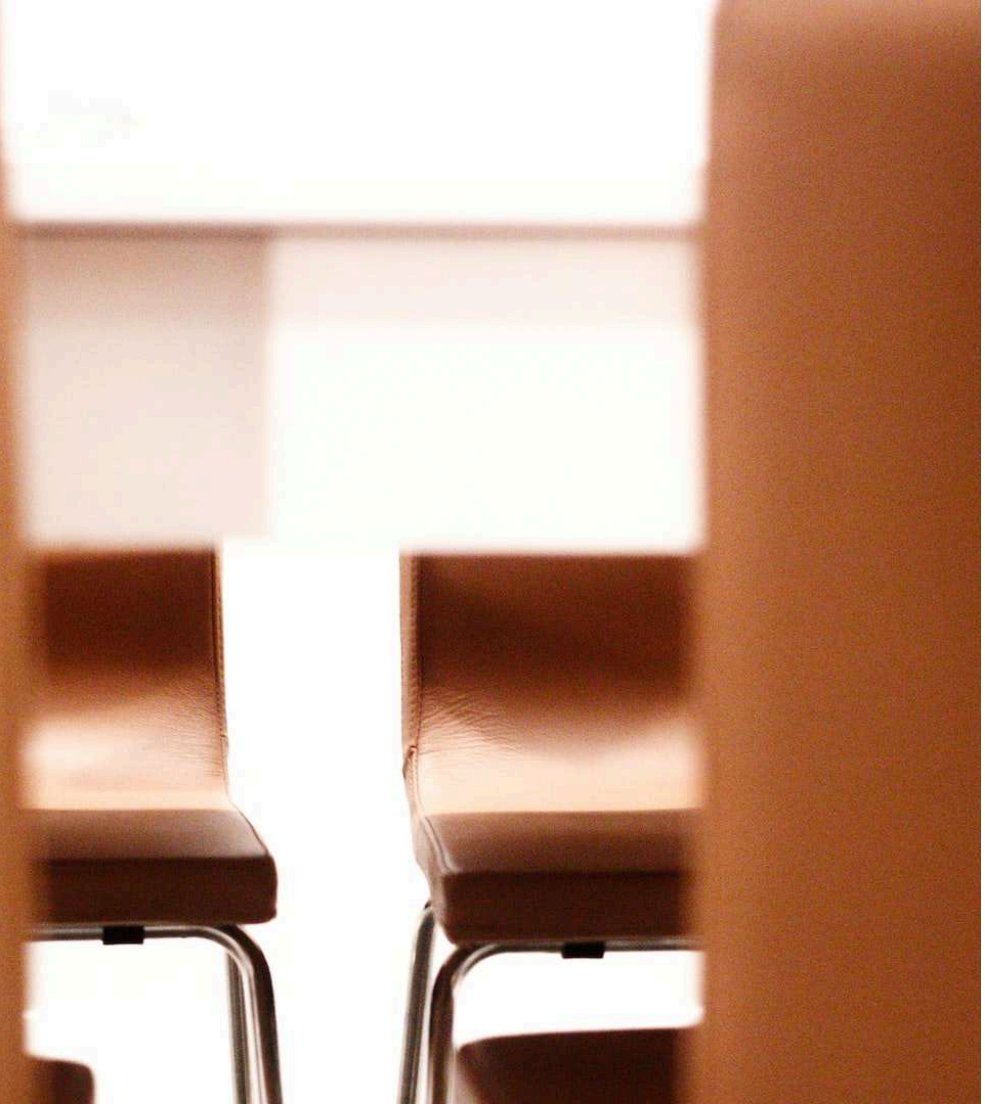
Efektywne tworzenie aplikacji webowych z

Angular





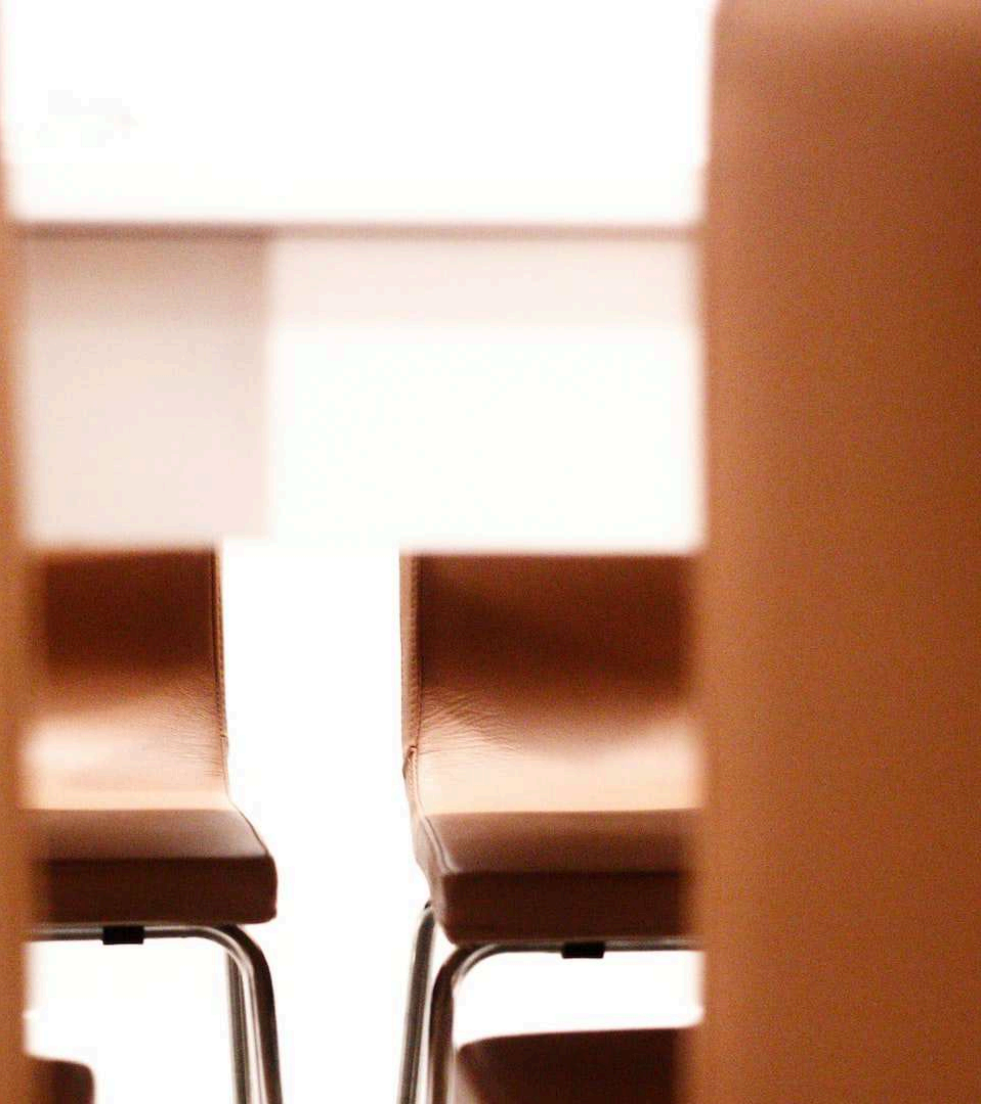
Szkolenie

- Plan szkolenia i cele
- Aktualne obowiązki
- Pytania, dyskusje, potrzeby
- Elastyczny program szkoleniowy



Mateusz Kulesza

- Senior Software Developer
- Team Leader
- Project Manager
- Consultant and Trainer



Uczestnik

czyli "Ty"

Kilka szybkich pytań:

- Twoje codzienne obowiązki?
- Technologie i background?
- Poprzednie doświadczenie?
- Twoje cele na to szkolenie?
- Określone obszary lub tematy?

Instalacje

Edytor

- Visual Studio Code
- InteliJ WebStorm

W terminalu sprawdzamy: Node, NPM i GIT

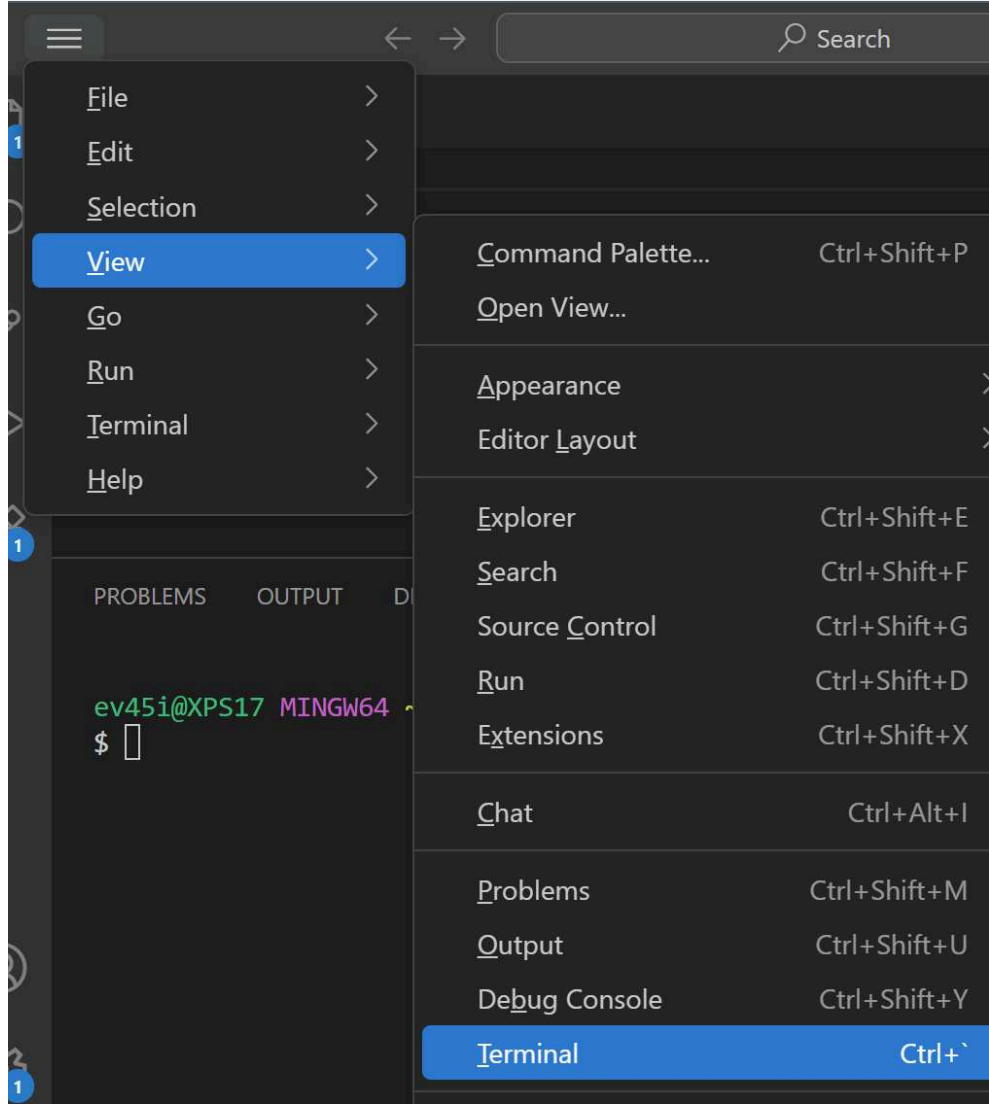
```
node -v
npm -v
git -v
```

Instalatory

- https://nodejs.org/en/download
- https://git-scm.com/downloads/win

Alternatywnie NVM by zmieniać wersje Node:

- https://github.com/coreybutler/nvm-windows



Narzędzie @angular/cli

Generator uruchamiany z wiersza poleceń

Instalacja:

```
npm install -g @angular/cli@latest
```

```
ng version
ng --help
```

```
ng new --help
ng s --help
```

https://angular.dev/tools/cli

```
Generates and/or modifies files based on a schematic

Commands:
  ng g <schematic>              Run the provided sche
  ng g ai-config                Generates AI configur
                                practices, improving
  ng g app-shell                Configures your proje
  ng g application [name]       Generates a new Angul
                                root component, modul
                                testing.
  ng g class [name]             Creates a new class i
                                a blueprint for creat
                                optional test files.
  ng g component [name]         Creates a new Angular
                                class, an HTML templa
  ng g config [type]            Generates configurati
                                compatibility. This s
  ng g directive [name]         Creates a new directi
                                allow you to manipula
                                code for a new direct
  ng g enum [name]              Creates a new enum in
                                maintainable. This sc
  ng g environments             Generates and configu
                                various environments,
                                to customize your app
  ng g guard [name]             Creates a new route g
```

ng new <projectname>

Generatory kodu

Stworzenie nowego projektu angular:

```
ng new nazwaprojektu
ng new nazwaprojektu --standalone
```

Uruchomienie lokalnego serwera developerskiego

```
ng serve
```

Wygenerowanie szkieletu komponentu:

```
ng generate component <name>
```

Generatory kodu to tzw. "Schematics"

```
$ ng new test
✓ Do you want to enable Server-Side Rendering (SSR)
Generation (SSG/Prerendering)? Yes
✓ Do you want to create a 'zoneless' application wi
✓ Which AI tools do you want to configure with Angu
https://angular.dev/ai/develop-with-ai None, GitHub
https://code.visualstudio.com/docs/copilot/copilot-
CREATE test/angular.json (2822 bytes)
CREATE test/package.json (1348 bytes)
CREATE test/README.md (1526 bytes)
CREATE test/tsconfig.json (1026 bytes)
CREATE test/.editorconfig (331 bytes)
CREATE test/.gitignore (647 bytes)
CREATE test/tsconfig.app.json (464 bytes)
CREATE test/tsconfig.spec.json (422 bytes)
CREATE test/.vscode/extensions.json (134 bytes)
CREATE test/.vscode/launch.json (490 bytes)
CREATE test/.vscode/tasks.json (980 bytes)
CREATE test/src/main.ts (228 bytes)
CREATE test/src/index.html (303 bytes)
CREATE test/src/styles.scss (81 bytes)
CREATE test/src/main.server.ts (300 bytes)
CREATE test/src/server.ts (1677 bytes)
CREATE test/src/app/app.spec.ts (685 bytes)
CREATE test/src/app/app.ts (299 bytes)
CREATE test/src/app/app.scss (0 bytes)
CREATE test/src/app/app.html (20464 bytes)
CREATE test/src/app/app.config.ts (541 bytes)
CREATE test/src/app/app.routes.ts (80 bytes)
CREATE test/src/app/app.config.server.ts (438 bytes
CREATE test/src/app/app.routes.server.ts (174 bytes
CREATE test/public/favicon.ico (15086 bytes)
CREATE test/.github/copilot-instructions.md (1983 b
⠸ Installing packages (npm)...
```

ng serve --open

Otwórz w przeglądarce:

http://localhost:4200/

Lokalny server developerski

- Szybkie przebudowanie kodu
- Odświeżanie przeglądarki
- HMR - "wsytrzykiwanie" kodu do aplikacji
- Dokładnejsze kody błędów

h + enter – menu developerskie

```
press r + enter to force reload browser
press u + enter to show server url
press o + enter to open in browser
press c + enter to clear console
press q + enter to quit
```

```
ng s -o
Browser bundles
Initial chunk files   | Names           |  Raw size
main.js               | main            |  50.75 kB |
styles.css            | styles          |  96 bytes |
polyfills.js          | polyfills       |  95 bytes |

                      | Initial total   |  50.94 kB


Server bundles
Initial chunk files   | Names           |  Raw size
main.server.mjs       | main.server     |  52.29 kB |
server.mjs            | server          |   1.79 kB |
polyfills.server.mjs  | polyfills.server| 266 bytes |

Application bundle generation complete. [2.041 seconds]

Watch mode enabled. Watching for file changes...
NOTE: Raw file sizes do not reflect development server
  → Local:   http://localhost:4200/
  → press h + enter to show help
```

Zasoby Angular

https://angular.dev/

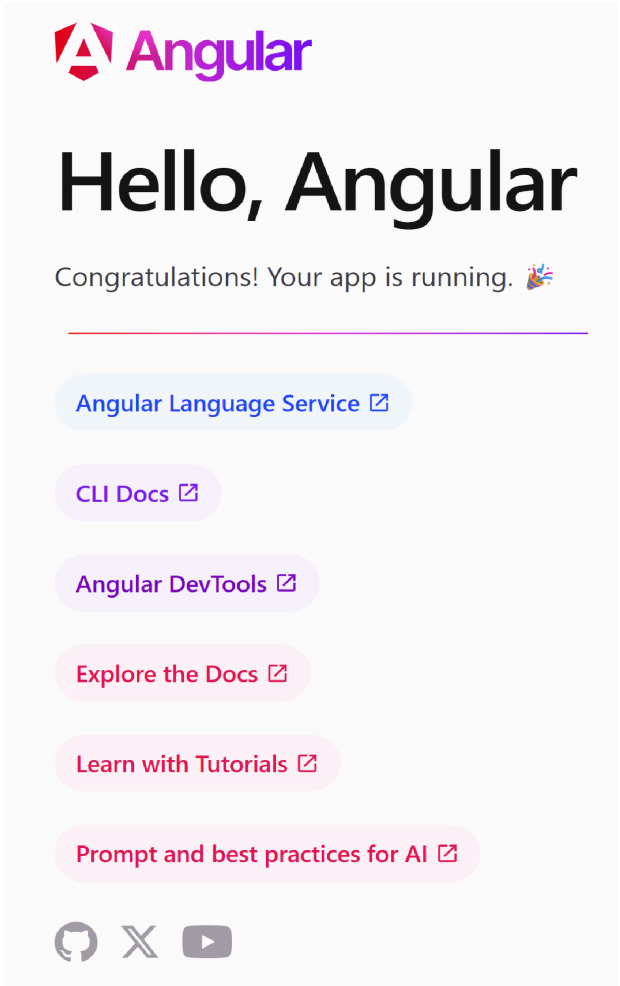
- https://angular.dev/tools/cli
- https://angular.dev/tools/language-service

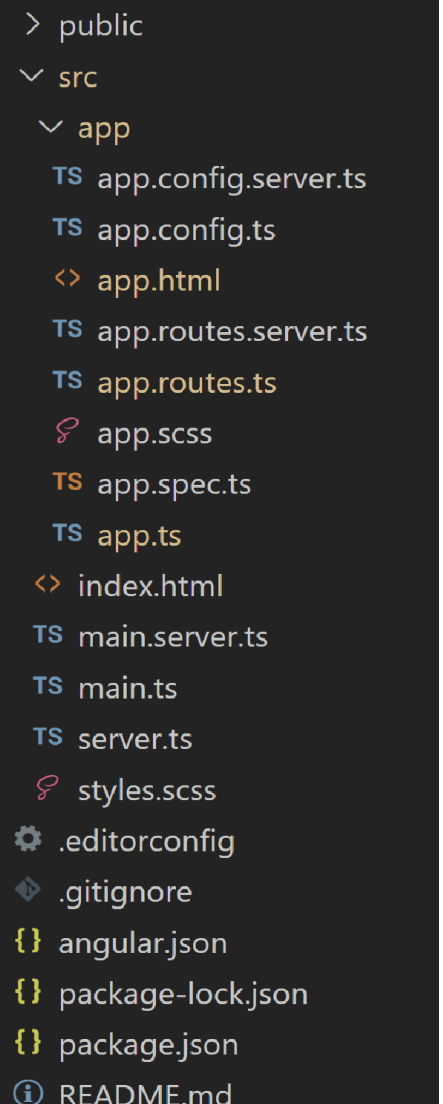
Tutorials

- first-app
- signals
- deferrable-views
- develop-with-ai

Devtools

- https://angular.dev/tools/devtools
- Devtools in Chrome WebStore





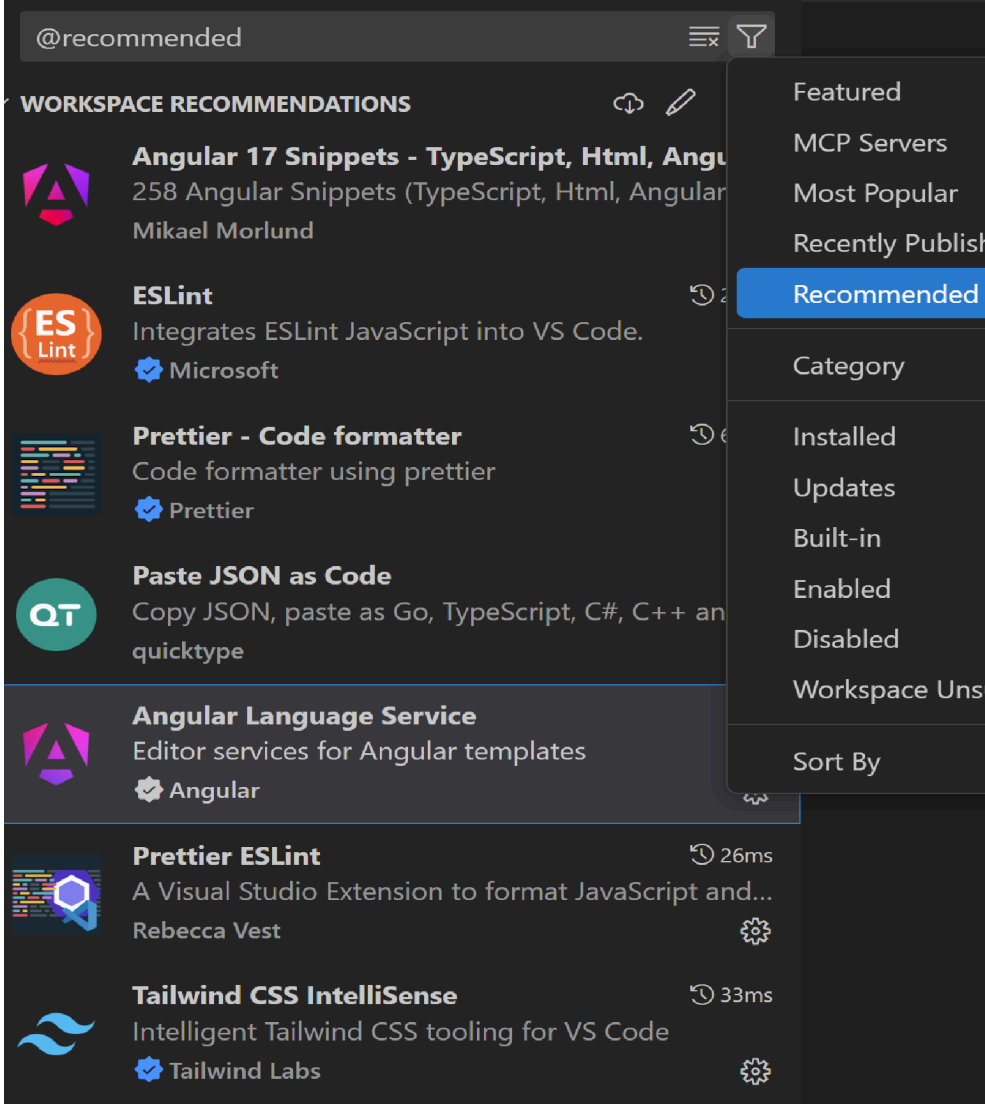
Struktura Plików

- **node_modules/** - Zainstalowane zależności
- **public/** - Publiczne zasoby statyczne
- **src/** - Kod źródłowy aplikacji
  - **index.html** - Główny plik HTML aplikacji
  - **main.ts** - Plik startowy po stronie klienta (entry point)
  - **app.config.ts** - Wspólna konfiguracja aplikacji
  - **app.ts** - Logika głównego komponentu Angulara
  - **app.html** - Główny szablon HTML komponentu aplikacji
  - **app.routes.ts** - Konfiguracja routingu po stronie klienta
  - **app.scss** - Globalne style SCSS dla komponentu aplikacji
  - **styles.scss** - Globalne style dla całego projektu
- **.editorconfig** - Zasady formatowania kodu dla edytorów
- **.gitignore** - Pliki i katalogi ignorowane przez Git
- **angular.json** - Konfiguracja projektu Angular
- **package-lock.json** - Dokładne wersje zależności (plik blokady npm)
- **package.json** - Informacje o projekcie, skrypty i lista zależności

Rozszerzenia

Visual Studio Code (Ctrl+Shift+X)

```
// .vscode/extensions.json
{
  // For more information, visit:
  // https://go.microsoft.com/fwlink/?linkid=827846
  "recommendations": [
    "angular.ng-template",
    "mikael.angular-beastcode",
    "dbaeumer.vscode-eslint",
    "rvest.vs-code-prettier-eslint",
    "esbenp.prettier-vscode",
    "yoavbls.pretty-ts-errors",
    "bradlc.vscode-tailwindcss",
    "quicktype.quicktype"
  ]
}
```



Tailwind

https://tailwindcss.com/

```
npm i tailwindcss @tailwindcss/postcss postcss --force
```

.postcssrc.json

```
{
  "plugins": {
    "@tailwindcss/postcss": {}
  }
}
```

app.html:

```
<h1 class="text-3xl font-bold underline">
  Hello world!
</h1>
```

```
@import "tailwindcss";

/* Input styles */
input {
  @apply px-3 py-2 border border-gray-300 rounded-md focu
}

/* Checkbox styles */
checkbox {
  @apply w-4 h-4 text-blue-600 bg-gray-100 border-gray-30
}

/* Textarea styles */
textarea {
  @apply px-3 py-2 border border-gray-300 rounded-md focu
}

/* Button styles */
button {
  @apply px-4 py-2 font-medium rounded-md border border-g
}

/* Primary button styles */
button[primary] {
  @apply px-4 py-2 font-medium rounded-md bg-blue-600 tex
}
```

Angular Material

https://material.angular.dev/

```
ng add @angular/material
```

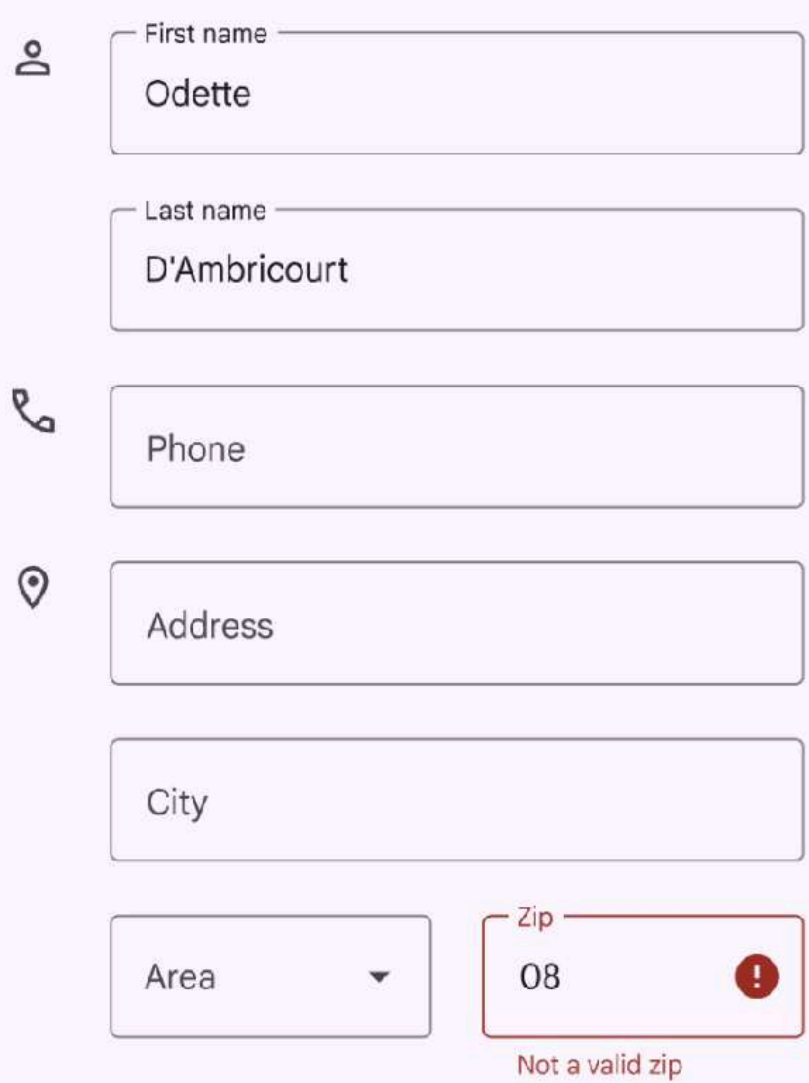
app.ts

```
import {
  MatSlideToggleModule
} from '@angular/material/slide-toggle';

@Component ({
  imports: [
    MatSlideToggleModule,
  ]
})
class AppComponent {}
```

app.html

```
<mat-slide-toggle>Toggle me!</mat-slide-toggle>
```



Bootstrap

Startowanie Applikacji

```
import { platformBrowserDynamic } from "@angular/platform-browser-dynamic";
// import { MyAppModule } from "./app/app.module";
import { AppComponent } from "./app/app.component";
```

Inicjalizacja głównego modułu (z wybranym głównym komponentem):

```
// platformBrowserDynamic().bootstrapModule(MyAppModule);
bootstrapApplication(AppComponent, appConfig)
```

Element głównego komponentu musi oczywiście znajdować się w dokumencie HTML, np.

```
<my-app> Loading… </my-app>
```

Moduły w Angular

@NgModule

```
import { NgModule } from "@angular/core";
import { BrowserModule } from "@angular/platform-browser";
import { AppComponent } from "./app.component";

@NgModule({
  imports: [BrowserModule],       // Importujemy “ exporty ” z innych modułów:

  declarations: [AppComponent],   // Deklaracje dla parsera HTML ( Dyrektywy i komponenty ):

  bootstrap: [AppComponent],      // Który komponent będzie “głównym” komponentem aplikacji:

  exports: [],                    // Udostępnione elementy

  providers: [],                  // definicje usług ( o tym później… )

})
export class MyAppModule {}
```

Standalone - Aplikacje "bezmodułowe"

main.ts

```
import { bootstrapApplication } from "@angular/platform-browser";
import { appConfig } from "./app/app.config";
import { AppComponent } from "./app/app.component";

bootstrapApplication(AppComponent, appConfig).catch((err) ⇒
  console.error(err)
);
```

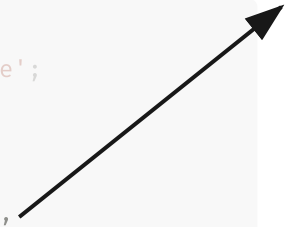
app.component.ts:

```
@Component({
  selector: "app-root",       // Selektor HTML
  // standalone: true,        // Bezmodułowy
  imports: [CommonModule, RouterOutlet, MyModule],
  templateUrl: "./app.component.html",
  styleUrls: ["./app.component.scss"],
})
export class AppComponent {
  title = "project";
}
```

Deklaracja komponentu

```
import { Component } from '@angular/core';

@Component({
  // standalone: true
  selector: 'app-root, .extra-selector',
  templateUrl: 'app.component.html',
  styleUrls: ['app.component.css']
})
export class AppComponent {
  title = 'app works!';
}
```



Selektor to prosty selektor CSS, który określa na nakich elementach HTML parser ma zamontować ten komponent

Na przykład podpinanie do:

- `[my-attribute]` - atrybutu
- `.spiner-loader` - klasy CSS
- `app-root` - elementu

Szablon

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  template: `<div>
      <h1>{{title}}</h1>
  </div>`
})
export class AppComponent {
  title = 'app works!';
}
```



Widok

```
<app-root>
  <div>
    <h1>app works!</h1>
  </div>
</app-root>
```

"Szablon" umieszcza "widok" w miejscu określonym przez "selektor"

Interpoluje wyrażenia umieszczone w klamrach

Enkapsulacja Styli

```
import { Component } from '@angular/core';
@Component({
  selector: 'app-root',
  templateUrl: 'app.component.html',
  encapsulation: ViewEncapsulation.ShadowDom,
  styles: [
  `h1{ color: red; }`,
  `p{ cursor: pointer; }`]
})
export class AppComponent {
  title = 'app works!';
```

Przebijanie enkapsulacji:

```
:host ::ng-deep h3 {
  font-style: italic;
}
```

Enkapsulacja może przyjąć wartości:

- **Emulated** (domyślnie) - style z dokumentu HTML propagowane są do komponentu i jego dzieci. Style zdefiniowane w @Component() nie wpływają na reszte dokumentu - są izolowane
- **ShadowDom**- Style dokumentu nie wpływają na elementy w komponencie, ani vice versa - pełna izolacja (patrz. shadow-dom)
- **None** - style są propagowane do dokumentu HTML, więc są dostępne dla pozostałych komponentów

Zagnieżdżone komponenty

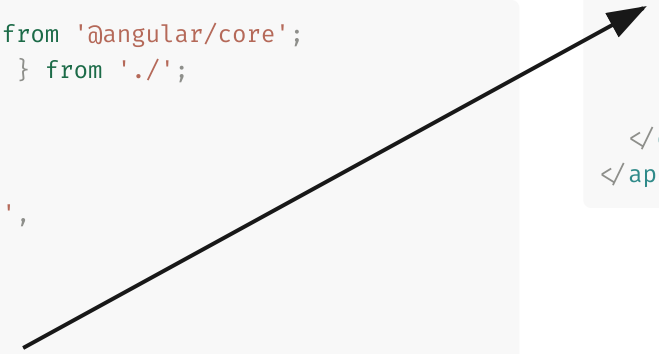
```
import { Component } from '@angular/core';
import { SubComponent } from './';


@Component({
  selector: 'app-root',
  template: `<div>
    <h1>Parent</h1>

    <sub-component/>

  </div>`
})
export class AppComponent { … }


@Component({
  selector: 'sub-componentt',
  template: `<div> <h3>Child</h3>
  </div>`
})
export class ChildComponent { … }
```



```
<app-root>
  <div>
    <h1>Parent!</h1>

    <sub-component>
      <h3>Child!</h3>
    </sub-component>

  </div>
</app-root>
```

Wiązania właściwości i dyrektywy atrybutów

Wiązania Własności

```
<div [id]=" ' identyfikator_' + myFakeUuid " [title]="zmienna">

<div title="Hello {{userName}}">
```

Style

```
<p [style.backgroundColor]="'lime'"> Jestem limonkowy! </p>
<p [style.background-color]="'lime'"> Jestem limonkowy! </p>
```

Jednostki

```
<p [style.font-size.px]="big? 24 : 12"> {{ big? 'duży' : 'mały' }}</p>
```

Klasy

```
<p [class.promotion]="true"> Tu Promocja! </p> ⟶ <p class=”promotion”> Promocja! </p>

<p [ngClass]="{ promotion: false, highlight:true }"> Tu nie ... </p> ⟶  <p class=”highlight”> Tu nie.. </p>
```

Lokalne Referencje

Umieszczenie znaku hash z nazwą (“#nazwa”) na elemencie tworzy “lokalną referencje”

To zmienna dostępną tylko w tym komponencie, która zawiera odniesienie do elementu DOM

(lub do komponentu)

```
<video #movieplayer …>…</video>


<button (click)="movieplayer.play()">
```

Można się do niej odwołać z kodu komponentu poprzez ViewChild

```
class AppComponent{

  @ViewChild('movieplayer')
  movieplayer?: ElementRef<HTMLVideoElement>

  ngAfterViewInit(){
    this.movieplayer
  }

}
```

Detekcja zmian w Angular

Jak to działa naprawdę

- Angular nie obserwuje danych (jak np. VueJS)
- Angular reaguje na zdarzenia użytkownika i asynchroniczne akcje
- Każde takie zdarzenie uruchamia globalny cykl Change Detection (CD)
- CD przechodzi od góry do dołu drzewa komponentów, aktualizując szablony

Bardziej "ziarniste" wykrywanie jest możliwe dzięki `angular.signals()`

Co uruchamia Change Detection

- click,
- input,
- setTimeout,
- promise,
- fetch,
- itp.

Po każdym takim zdarzeniu:

```
(click)
    → tick()
        → AppComponent
            → dzieci
                → wnuki  ...
Zone.js
    → Angular.tick()
        → sprawdź drzewo komponentów
```

Strategie detekcji zmian

OnPush - co to znaczy

```
@Component({
    changeDetection: ChangeDetectionStrategy.OnPush
})
```

- OnPush wyłącza komponent z globalnego sprawdzania zmian
- OnPush zakłada, że dane są niemutowalne
- Angular reaguje tylko, gdy @Inputt / input() zmieni referencję
- Możemy wymusić ręcznie dzięki `ChangeDetectorRef`
- Komponent nasłuchuje także zmian na `angular.signal()`

Strategia	Opis
`Default`	Sprawdzaj wszystko - Każdy tick przechodzi całe drzewo
`OnPush`	Sprawdzaj gdy zmienią się wejścia - Komponent i dzieci “zakładają słuchawki”



Komponent z OnPush

zostanie sprawdzony, gdy:

- Zmieni się referencja w @Input / input()
- Wywołasz markForCheck() (oznaczenie “do sprawdzenia” w górę)
- Wywołasz detectChanges() (sprawdzenie natychmiast, w dół)
- OnPush = komponent sam decyduje, kiedy chce być odświeżony
- `ChangeDetectorRef` — precyzyjna kontrola

```
class Component {
    constructor(private cdr: ChangeDetectorRef) {}

    cdr = inject(ChangeDetectorRef)
```

- `markForCheck()`
  - oznacza komponent i **jego przodków**
  - “w górę”
  - **asynchronicznie** (przy następnym ticku)
- `detectChanges()`
  - uruchamia CD dla **komponentu i jego dzieci**
  - “w dół”
  - **natychmiast** (synchronnie) |

Przykład:

```
// sprawdź natychmiast mnie i dzieci
this.cdr.detectChanges();

// powiadom Angulara: "sprawdź mnie w następnym ticku"
this.cdr.markForCheck();
```

Lifecycle hooks a CD

Hook	Kiedy działa	Uwagi
`ngOnChanges()`	przy zmianie @Input()	tylko przy nowej referencji
`ngDoCheck()`	przy każdym ticku	można nadpisać logikę sprawdzania
`ngAfterViewChecked()`	po zakończeniu CD dla widoku	używać ostrożnie

> ngDoCheck stosować tylko gdy Angularowy mechanizm nie wystarcza. To punkt “ręcznej interwencji”.

“ExpressionChangedAfterItHasBeenCheckedError”

Typowy błąd w CD

- Wartość zmieniła się po tym, jak Angular już sprawdził dany komponent
- Najczęściej: zmiana stanu w ngAfterViewInit() lub asynchronicznie w trakcie CD

```
ngAfterViewInit() {
    // ❌ Angular już sprawdził widok
    this.title = 'Nowy';

    //  ❌ Jesteśmy już głębiej w drzewie
    this.parent = 'Cofamy się ... '
}
```

Naprawa:

- setTimeout() = Zone.js -> next tick()
- detectChanges() = Nowy cykl
- signal.set() => next tick()

```
ngAfterViewInit() {
    this.title = 'Nowy';

    // ✅ wymuś ponowne sprawdzenie
    this.cdr.detectChanges();

    // ✅ lub signal - Zmiana w kolejnym tick()
    this.parentSignal.set('Nowy')
}
```

Zone.js i NgZone

Zone.js “owija” wszystkie asynchroniczne API przeglądarki

> Po każdym async → automatyczny tick Angulara

Można uciec poza Angulara:

```
ngZone = inject(NgZone)

this.ngZone.runOutsideAngular(() ⇒ {
    window.addEventListener('scroll', () ⇒ {
        // brak CD, nie spowoduje odświeżenia
    });
});
```

A gdy potrzebujesz odświeżenia:

```
    this.ngZone.run(() ⇒ this.cdr.markForCheck());
```

Zoneless Angular i Signals

- Nowoczesny Angular (v17+) może działać bez Zone.js
- Zamiast globalnych ticków — precyzyjna reactivity przez Signals
- Każdy komponent “subskrybuje” tylko te sygnały, z których korzysta
- Gdy sygnał się zmienia → komponent renderuje się ponownie
- Jeśli sygnał przestaje być używany w szablonie → Angular automatycznie “odsubskrybowuje”

Signals

a Change Detection

- W modelu sygnałów nie ma globalnego cyklu CD
- Zmiana sygnału aktualizuje powiązany widok
- Nie ma więc momentu, w którym “wartość zmieniła się po sprawdzeniu”
- Każda aktualizacja sygnału to:
  - aktualizacja widoku w tym samym przebiegu
- Błąd “ExpressionChangedAfter...” znika całkowicie

Jak komponent śledzi sygnały

Podczas renderowania szablonu Angular::

- rejestruje które sygnały zostały odczytane
- dodaje reakcję - jeśli ten sygnał się zmieni to oznacz ten komponent jako **"do odświeżenia"**

- **Przy kolejnym renderze:**
  - **"brudne"** komponenty przechodzą Change Detection
  - sygnały nieużyte zostają automatycznie odsubskrybowane

- **Dzięki temu CD jest lokalne i precyzyjne:**
  - tylko komponenty zależne od zmienionych sygnałów są renderowane.

Podsumowanie

- Angular nie obserwuje danych, tylko akcje.
- Zone.js
  - automatyczny CD po każdym zdarzeniu
- OnPush
  - kontrola zakresu i momentu sprawdzania
- detectChanges() -
  - sprawdź synchronicznie, w dół
- markForCheck()
  - oznacz w górę, sprawdź w dół (async)
- Signals
  - eliminują globalny CD i błędy z kolejnością sprawdzeń
  - Komponent “śledzi” tylko używane sygnały i reaguje lokalnie



Angular Signals

— Nowy model reaktywności

- Precyzyjny. Lokalny. Deterministyczny.
- Zastępuje globalną detekcję zmian
- Komponent reaguje tylko na użyte sygnały

Nie ma potrzeby:

- Zone.js
- ticków
- ani “ExpressionChangedAfter...”

Przechowuje wartość + pozwala ją odczytać i obserwować

---

signal() — reaktywna wartość

```
import { signal } from '@angular/core';

const count = signal(0);
```

Do odczytu — funkcja:

```
console.log(count()); // 0
```

Zmiana wartości:

```
count.set(5);
```

Aktualizacja wartości:

```
count.update(prev ⇒ prev + 5);
```

signal.set(value) vs signal.update(fn)

Metoda	Działanie	Kiedy używać
`set(value)`	nadpisuje wartość	gdy masz **konkretną nową wartość**
`update(fn)`	oblicza nową na podstawie starej	gdy chcesz **zaktualizować inkrementalnie**

```
count.set(10);               // ustaw na 10
count.update(v ⇒ v + 1);     // zwiększ o 1
```

> update działa transakcyjnie – Angular wykona wszystkie reakcje po zakończeniu aktualizacji.

computed()

— wartości pochodne

```
import { computed } from '@angular/core';

const count = signal(2);
const double = computed(() => count() * 2);

console.log(double()); // 4
```

- Oblicza wartość na podstawie innych sygnałów
- Automatycznie reaguje, gdy zależności się zmienią
- Nie można w nim wywoływać set ani update
- computed = czysta funkcja zależna od innych sygnałów

effect()

— efekty uboczne

```
fieldName = effect(() => {
  console.log(
    `The current count is: ${doubleCount()}`
  );
});
```

- Uruchamia się natychmiast i przy każdej zmianie użytych sygnałów
- Służy do skutków ubocznych (API calls, logi, DOM itp.)
- Nie zwraca wartości
- effect = obserwator, computed = pochodna

effect()

— reagowanie na zmiany

```
import { effect } from '@angular/core';

effect(() => {
  console.log('Count changed:', count());
});
```

- Każdy efekt może zwrócić funkcję “cleanup”, która wykona się:
  - przed kolejnym uruchomieniem efektu
  - przy zniszczeniu efektu (np. usunięciu komponentu)

onCleanup()

```
effect((onCleanup) => {
  const id = setInterval(() => {
    console.log(count())
  }, 1000);
  onCleanup(() => clearInterval(id));
});
```

> Podobnie jak useEffect w React — ale deterministycznie.

debounce z effect()

- setTimeout() i cleanup

```
const text = signal('');
const result = signal('');

effect((onCleanup) => {
  const t = setTimeout(() => {
    result.set('Wynik dla: ' + text());
  }, 500);

  onCleanup(() => clearTimeout(t)); // debounce
});
```

- Każda zmiana text restartuje timeout
- Efekt nie “nadrabia” starych zmian
- Czyste, bez Subject, switchMap i RxJS

Kiedy co stosować ?

Typ	Co robi	Kiedy używać
`signal()`	wartość źródłowa	stan lokalny, dane formularza, interakcje
`signal.set()`	przypisanie nowej wartości	bezpośrednia zmiana (np. reset, API)
`signal.update()`	transformacja na podstawie starej	inkrementacje, toggle, mutacje
`computed()`	wartość pochodna	zależne pola, filtrowane dane
`effect()`	reakcja z efektem ubocznym	logi, fetch, DOM, debounce, subskrypcje

Wiązanie Właściwości

Komunikacja do komponentu zagnieżdżonego

```
@Component({
 selector: 'todo-input',
 template: '...'
})

export class Hello {
  @Input() title = '';

  @Input() item?: MyTodoType;

  // Wejście bez wartości domyślnej
  @Input() optional?: string

  // Angular 16+ - wymagane wejście
  @Input({required:true}) someProp!: Type

  // ———————————————————————
  // Angular 18+  - Signal input
  newInput = input<MyTodoType>()

}
```


Przekazywanie wartości jako statyczny tekst

```
<todo-input title="Kupić chleb"></todo-input>
```

Przekazywanie przez dynamiczne jako wyrażenie

```
<todo-input [item]="getMyItem()"></todo-input>
```

Emiter zdarzeń

Komunikacja do komponentu nadrzędnego

```
@Component({
 selector: 'todo-input',
 template: '<button (click)="clickCompleted()"> ..
})
export class TodoInput {
 // Angular 2+
 // @Output() completed = new EventEmitter<TodoType>();

 // Angular 18+  - Signal output
 newOutput = output<MyTodoType>()

 clickCompleted(){
  // kliknięcie wysyła sygnał do rodzica
  this.completed.emit( this.todo )
 }
}
```



```
@Component({
 selector: 'todo-list',
 template: `
  <todo-input (completed)="saveProgress($event)" />
 `
})
export class TodoList {

  saveProgress(todo:Todo){
    // ... funkcja wykonana przez podkomponent
  }

}
```

Komunikacja z komponentem w obie strony

Dyrektywa ngModel:

Wiąże dane ze zmienną:

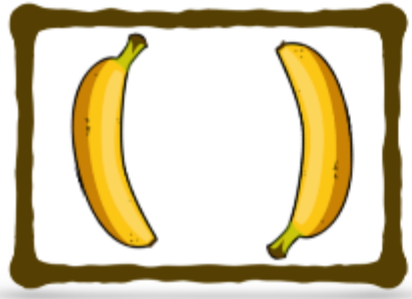
```
<input [ngModel]="name">                                    <——— tylko aktualizacja wartości
```

Wiąże nasłuchiwanie zdarzeń zmian (onChange):

```
<input ngModel (ngModelChange)=" name = $event ">           <——— tylko nasłuch zmian
```

Lub w skrócie:

```
 <input [(ngModel)]="name">                                 <——— Dwustronne wiązanie danych ( aktualizacja i nasłuch )
```



[(ngModel)]

Dyrektywy strukturalne

- nglf - Dodaje i usuwa element szablonu (template) jeśli wyrażenie jest prawdziwe

```
<ng-template [ngIf]="condition">
 <div>{{ name }}</div>
</ng-template>
```

Wersja skrócona:

Oznaczenie “ * “ przed dyrektywą tworzy szablon za nas

```
<div *ngIf="condition">
  <p> {{ name }}  </p>
</div>
```

Dyrektywy strukturalne

Pozwalają modyfikować strukturę DOM

```
<div *ngIf="completed"> ✓ </div>
```

```
<div *ngFor="let item of todoList; let i = index">
   ... It gets repeated often ...
</div>
```

```
<div [ngSwitch]="item.status">
 <p *ngSwitchCase="'completed'"> Completed <p>
 <p *ngSwitchCase="'in-progress'">
    Working on it
  <p>
 <p *ngSwitchDefault> Working on it <p>
</div>
```



Dyrektywy

Angular 2+ Syntax

```
<div *ngIf="getUser() as user; else guestMessage">
  The user {{user}} is logged in
</div>

<ng-template #guestMessage>
  The user is not logged in
</ng-template>
```

```
<div *ngFor="let item of items; track: item.id">
```

```
  {{ item.name }}
</div>

<ng-container [ngSwitch]="modes">
  <ng-container *ngSwitchCase="'modeA'"> ...
  <ng-container *ngSwitchCase="'modeB'"> ...
  <ng-container *ngSwitchDefault> ...
```

Kontrola przepływu

Angular 17+ Syntax

```
@if ( getUser() as user;) {
  <p>The user  {{user}} is logged in</p>

} @else {
  <p> The user is not logged in</p>
}
```

```
@for (item of items; let i = $index; track item.id) {
  {{i+1}}. {{ item.name }}
}
```

```
@switch (condition)
{
  @case (caseA) {
    <p>Case A.</p>
  }
  @default {
    Default case.
  }
}
```

Projekcja zawartości

za pomocą dyrektywy ngContent możemy umiejscowić w szablonie komponentu html wpisany jako zawartość komponentu

```
@Component({
 selector: 'user-profile',
 template: `
  <h4>User profile</h4>
  <ng-content></ng-content>`
})
class Child {}
```

Opcjonalnie można zdefiniować miejsce dla elementu z określonego selektora :

```
<ng-content select=".my-css-selector">
```

```
<user-profile>
  <h5>Lito Rodriguez</h5>
  <small>Actor</small>
</user-profile>
```



```
<user-profile>
  <h4>User profile</h4>
  <h5>Lito Rodriguez</h5>
  <small>Actor</small>
</user-profile>
```

Cykl życia

Dyrektywy (D) i komponenty © mogą “wpiąć” się w cykl renderowania aby wykonać własne funkcje:



hook:	moment wystąpienia:
ngOnChanges *(C, D)*	when a data-bound input property value changes
ngOnInit *(C, D)*	after the first `ngOnChanges`
ngDoCheck *(C, D)*	during every Angular change detection cycle
ngAfterContentInit *(C)*	after projecting content into the component
ngAfterContentChecked *(C)*	after every check of projected component content
ngAfterViewInit *(C)*	after initializing the component's views and child views
ngAfterViewChecked *(C, D)*	after every check of the component's views and child views
ngOnDestroy *(C, D)*	just before Angular destroys the directive/component

Własne dyrektywy wiążące

```
@Directive({
 selector: '[myDecorations]'
})
export class MyDecorationsDirective{
    constructor(private el: ElementRef, private renderer: Renderer) { }

    // Nasłuchujemy zdarzeń na elemencie hosta:
    @HostListener('mouseenter') doMouseHighlight(){
       renderer.setElementStyle(
                    el.nativeElement, 'backgroundColor', 'yellow');
    }
    // Podstawiamy własne wartości do właściwości elementu hosta:
    @HostBinding('style.textDecoration') decoration = 'underline';
}
```

Własne dyrektywy strukturalne

```
@Directive({
  selector: '[myUnless]'
})
export class UnlessDirective {

  constructor(
    private templateRef: TemplateRef<any>,
    private viewContainer: ViewContainerRef
  ) { }

  @Input() set myUnless(condition: boolean) {
    if (!condition) {

      // Umieszczamy element z szablonu w widoku
      this.viewContainer.createEmbeddedView(this.templateRef);

    } else {

      // Usuwamy stworzone elementy:
      this.viewContainer.clear();

    }
  }
}
```

Usługi

i wstrzykiwanie zależności w Angular

Korzystanie z usług

Usługą jest każda instancja klasy, którą możemy przekazać do Komponentu, lub dyrektywy.

Usługi mogą:

- udostępniać przydatną logikę - np. komunikacja z serwerem
- przechowywać dane i współdzielić je pomiędzy komponentami
- informować komponenty o zdarzeniach i przekazywać potrzebne informacje
- i generalnie wszystkie inne zadania niezwiązane bezpośrednio z wyświetlaniem (Widokiem)

Nie tworzymy usług samodzielnie. Prosimy angulara o ich dostarczenie:

```
class ComponentOrService{
  constructor(
    private nazwa_zmiennej: KlasaUsługi,
    private inna: TypowanaUsługa ){
      this.inna // TypowanaUsługa
    }
```

Angular zajmie się dostarczeniem odpowiedniego obiektu!

Komunikacja HTTP

Komunikacja z API REST`owym i nie tylko z wykorzystaniem modułu angular/https

Moduł angular/http

```
@Component()
class ContactsApp implements OnInit {

  contacts:Contact[] = [];

  constructor(private http: HttpClient) {}

  ngOnInit() {
    // Http.get() - tworzy tzw. “zimny strumień”:
    this.http.get('/contacts')

      // Leniwe zapytania:
      //  Zapytanie do serwera wykona się
      //  dopiero przy subskrypcji:
      .subscribe(contacts ⇒ {
        this.contacts = contacts
      });
```

Nie używaj źródeł danych (m.in. Http) bezpośrednio w komponentach! Komponenty powinny być proste i zawierać tylko logikę widoku.

Takie powiązanie z Http uniemożliwia wymianę danych z innymi komponentami - Używaj usług!

Metody i konfiguracja http

Klasa Http posiada metody odpowiadające metodom HTTP: GET, POST, PUT, DELETE:

```
let options = new RequestOptions({
  headers: new HttpHeaders({
    // 'Content-Type': 'application/json'
  })
})

this.http
    .post(url, payload, options)
    .subscribe((data) => {
      console.log(data)
    })
```

Jeśli chcesz zmienić opcje dla całego modułu stwórz provider dla HttpInterceptor

Wstrzykiwanie zależności

Angular Dependecy Injection

Injektor zależności

Tworząc zależności wewnątrz naszych klas “Łączymy na sztywno” implementacje:

```
this.item = new Todo()
```

Lepiej jest wiązać klasy przy użuciu abstrakcji, a implementacje wstrzyknąć:

```
class Todo extends ListItem {}


export class InjectorComponent {
  item: ListItem = this.injector.get(ListItem);

  constructor(private injector: Injector) { }
```

Konfiguracja Providerów

Nie musimy ręcznie tworzyć injektorów! - już samo bootstrapModule(MyAppModule) tworzy za nas automatycznie globalny Injector.

OK, tylko jak wskazać angularowi której implementacji ma użyć ???

Domyślnie wystarczy zarejestować klasę, by injector sam zbudował dla nas obiekt tej klasy wraz z zależnościami:

```
@NgModule({
  // ...
  exports: [ NaszaKlasa ],
  providers: [ NaszaKlasa ],
})
export class MyAppModule { }
```

Klasa w sekcji providers będzie wstrzykiwana wszędzie gdzie użyto jej typu w tym module i w jego dzieciach

Automatyczne wstrzykiwanie

Co gdy nasza instancja wymaga innych instancji? Injektor zbuduje za nas cały graf:

Wystarczy dodać dekorator `@Injectable`:

```
@Injectable()
export class Nasza Klasa {
  constructor(@Inject(Logger) logger) {  }

  // lub wystarczy klasa/typ
  // - auto-wstrzykiwanie  po typie:
  constructor(private logger: Logger) {  }
}
```

Inne dekoratory: @Component, @Directive, @Pipe, itd. rozszerzają @Injectable()

Podmiana Implementacji

Gdy chcemy oddzielić implementacje od interfejsu i wstrzyknąć inny obiekt, mamy kilka możliwości:

```
@NgModule({
  // ...
  providers: [
    NazwaKlasy, // Ta sama implementacja…

    // Inna klasa
    { provide: Typ, useClass: Klasa },

    // Gotowa instacja, prosty obiekt,
    //  lub nawet wartość (np. config..)
    { provide: Typ, useValue: Obiekt },

    // Funkcja fabrykująca
    // - może mieć własne zależności ( deps:[...] )
    { provide: Typ, useFactory: ( someAPI ) ⇒ {
        // ... return someAPI.someSetup()

    // Zależności dla funkcji fabryki
    }, deps: [ SomeAPIClass ]},
  ],
})
```

Tokeny

Może się zdażyć, że wygodniejsze będzie użycie nazwy...

Nie używamy stringów, ale InjectionToken() - jest to unikalny symbol, dzięki czemu w przeciwieństwie do ciągów znaków unikamy ryzyka wystąpienia kolizji nazw!

```
const URL_TOKEN = new InjectionToken('URLToken');

@NgModule({
  providers: [
    {
      provide: URL_TOKEN,
      useValue: "http://example.com/api/v1/"
    },
    {
      provide: APIService,
      useFactory: (url) => {
        return new ApiService(url);
      }, deps: [ URL_TOKEN ]
    }
  ],
})
```

Hierarchiczny Injector

Domyślnie, każda stworzona instancja jest singletonem

Angular tworzy obiekt raz, a następnie przekazuje zawsze globalnie tą samą instancję.

Jeśli chcemy stworzyć lokalną instancje, definiujemy ją w `@Component`

```
@Component({
  selector: 'isolated-data-view'
  template: `<child-component/>`
  providers: [ NaszaUsluga ]
})
export class ParentComponent { }
```

```
class NaszaUsluga{
  // Usługa niszczona wraz z komponentem
  ngOnDestroy(){}
}
```

W module jest już zarejestrowana klasa “NaszaUsluga” i istnieje jej instancja...

Jednak ten komponent posiada teraz własny injektor, więc ten komponent i jego dzieci otrzymają nową instancję klasy NaszaUsluga niedostępną dla reszty komponentów.

Jeśli klasa nie jest zdefiniowana tutaj, będzie użyty injektor nadrzędny, nadrzędny, itd...

... aż po globalny injector.

```
export class ChildComponent{
  // Lokalna instancja z ParentComponent
  usluga = inject(NaszaUsluga)
}
```

Programowanie Reaktywne

Dzięki EventEmitter oraz rozszerzeniu Rx.JS możemy pracować na reaktywnych strumieniach

RxJS

czyli Reactive Extensions - “strumienie zdarzeń”



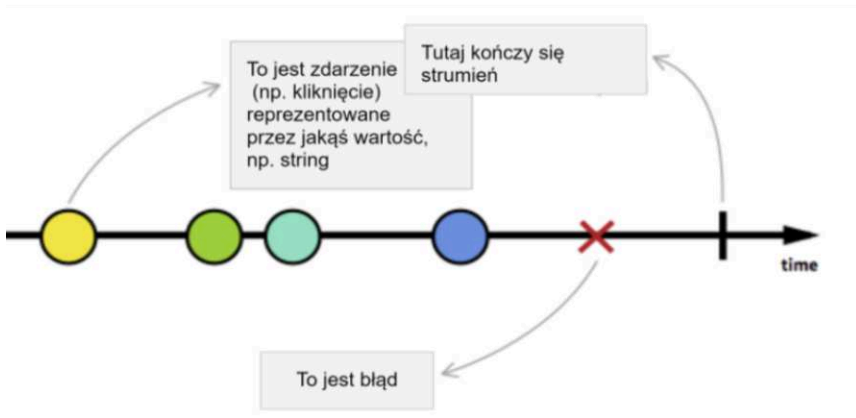
Każdy obiekt typu `EventEmitter<T>()` jest rozszerza `Observable`.

O strumieniach najprościej rozumować korzystając z diagramów kulkowych (marbles)

Zdarzeniami mogą być dowolne dane:

- Kliknięcia myszy,
- Znaki z klawiatury,
- Tyknięcie zegara,
- lub asynchroniczna odpowiedź z serwera ...

Zaletą strumieni jest możliwość ich wielokrotnego użycia



*Źródło: "Introduction to Reactive Programming you’ve been missing" by Andre Staltz

RxJS

RxJS posiada bogatą bibliotekę operatorów umożliwiających przekształcenia

Strumienie możemy

- obserwować,
- przekształcać

... a następnie subskrybować w celu otrzymania informacji, kiedy nastąpiła zmiana.

Sposób działania poszczególnych operatorów obrazuje się za pomocą "marble diagrams”

- https://rxjs.dev/guide/operators
- http://rxmarbles.com/
- https://www.learnrxjs.io/
- https://rxviz.com/

Operatory możemy podzielić na:

- przekształcające (np. delay, map, debounce, scan)
- łączące (np. merge, sample, startWith, zip)
- filtrujące (np. distinctUntilChanged, filter, skip)
- i wiele innych ...

Wynik działania operatora to strumień => można je łączyć

Zapis ASCII Marble:

```
--a---b-c---d---X-----|->

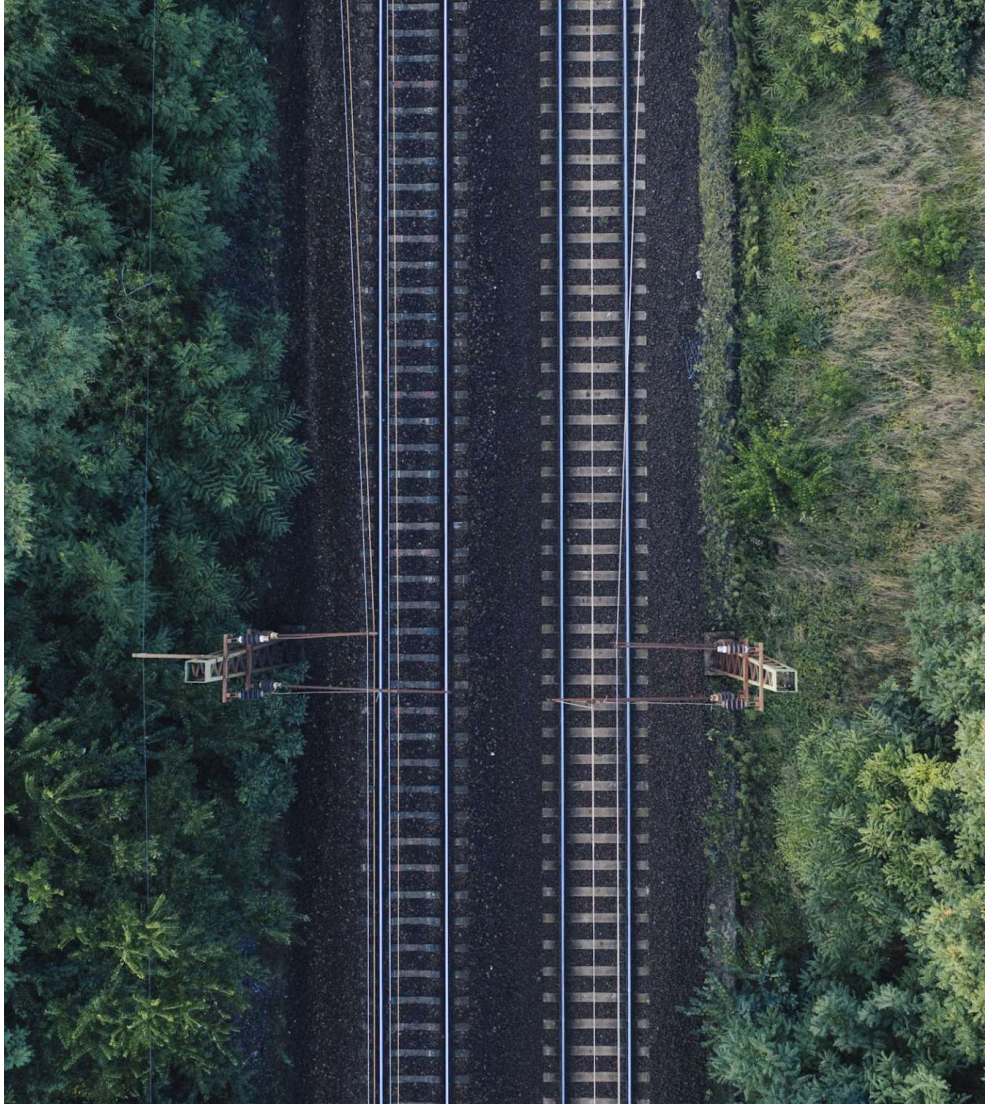
a, b, c, d są emitowanymi wartościami
X to błąd
| to sygnał „ukończony”
-----> to oś czasu
```

Rx Viz - https://rx-viz.vercel.app/

```
from([10, 20, 30]).pipe(
  delay(1000)
)
from([10, 20, 30]).pipe(
  delayWhen(() ⇒ 1000)
)
from([10, 20, 30]).pipe(
  delayWhen(() ⇒ timer(1000))
)
from([10, 20, 30]).pipe(
  delayWhen((x) ⇒ timer(x * 100))
)
from([40, 20, 30]).pipe(
  delayWhen((x) ⇒ timer(x * 100))
)

const mouse = fromEvent(document, 'click')

from([40, 20, 30]).pipe(
  delayWhen((x) ⇒ mouse)
)
from([40, 20, 30]).pipe(
  concatMap((x) ⇒ of(x).pipe(
    delayWhen((x) ⇒ mouse))
  )
```



catchError

```
pipe(
  catchError<T>((error, sourceObservable) ⇒ {
    throw new Error(error.message);
    return throwError(() ⇒ new Error(error.message));

    return sourceObservable // retry whole pipe above

    return this.http.get<Album[]>('inny_server').pipe()

    return from([[], []]);    // -00|⟼
    return of([]);            // —|⟼
    return [[], []];          // -00|⟼
    return from([]);          // —|⟼
    return [];                // —|⟼

    return EMPTY;             // —|⟼
    return NEVER;             // ————————⟶

}))
```

Zwracamy "strumienio-podobny" input:

```
export type ObservableInput<T> =
  | Observable<T>
  | InteropObservable<T>
  | AsyncIterable<T>
  | PromiseLike<T>
  | ArrayLike<T>
  | Iterable<T>
  | ReadableStreamLike<T>;
```

Każdy z tych typów zostanie zmieniony w observable i zasubskrybowany przez kolejny operator lub subscriber

Pamiętamy o unsubscribe

```
@Component({ ... })
class MyComponent{
  sub = new Subscription()

  ngOnInit(){
    sub.add(SomeObservable.subscribe( ... ))
  }

  ngOnDestroy(){
    this.sub.unsubscribe()
  }

  // Lub (Angular 16+)

  destroyRef = inject(DestroyRef);

  ngOnInit(){
   const sub2 = SomeObservable.subscribe( ... )
   this.destroyRef.onDestroy(() ⇒ sub2.unsubscribe());
  }

  // lub ( Angular 16+)
  sub3 = SomeObservable.pipe(takeUntilDestroyed())subscribe( ... )

}
```

RxJS w Angular 2

Angular 2 używa RxJS w kilku obszarach:

- EventEmitter
- API modułu Http
- Zmiany wartości formularzy to też strumienie
- Łatwa możliwość subskrypcji z poziomu widoku dzięki AsyncPipe

```
<div *ngFor="let item of todoListStream | async"> ... </div>
```

Możemy też tworzyć własne strumienie korzystając z obiektów:

- EventEmitter / Observable - do emitowania własnych zdarzeń
- Subject - do “przekazywania” stumieni pomiędzy usługami i komponentami

Formularze

Template Forms oraz Data-Driven Forms

Formularz reaktywny

```
import { FormsModule }   from '@angular/forms';
/* Importujemy Forms Module do Modułu Aplikacji ... */

import { FormControl, FormGroup, Validators } from '@angular/forms';

/* @Component ... */
export class MyForm {
  this.regForm = new FormGroup({
    username: new FormControl('wartość domyślna', [
          Validators.required, Validators.minLength(3)])
  });
```

lub

```
class MyForm{

  constructor(private builder: FormBuilder ) {
    this.builder.group({ username: […] })
    this.regForm.value // {username: 'Johny'}
  }
}
```

Łączenie z widokiem

Element FormGroup łączymy z formularzem dyrektywą FormGroup

Natomiast obiekt FormControl dyrektywą FormControlName:

```
<form [formGroup]="regForm"
      (ngSubmit)="saveUser(regForm)">

      <input name="username"
             formControlName="username" />

      <button type="submit">Save</button>
</form>
```

Dzięki podaniu formControlName możemy odwoływać się do pól formularza po tej nazwie:

```
this.regForm.get("username").value
```

Stany formularza

Formularz i jego pola mogą być w kilku stanach stanach

stan:	znaczenie:
pristine	pole nie było modyfikowane
dirty	pole było modyfikowane
touched	pole zostało opuszczone (blur)
valid	żaden walidator nie zwrócił błędu
submitted	formularz został wysłany

Klasy CSS

w formularzach

Elementy formularza otrzymują również odpowiednie klasy:

Klasa	Znaczenie
ng-pristine	pristine = true i dirty = false
ng-dirty	pristine = false i dirty = true
ng-touched	touched = true
ng-valid	valid = true
ng-invalid	valid = false

Możemy je więc wykorzystać w CSS:

```
input.ng-invalid.ng-dirty {
 border-bottom-color: red;
}
```

Własne metody walidacji

Walidator to funkcja przyjmująca jako parametr instancję pola (Control) i zwracającą obiekt z kluczami będącymi kodami błędów oraz wartościami boolean jeśli dana wartość jest błędna.

```
function startsWithLetter(control: Control): {[key: string]: any} {
    let pattern: RegExp = /^[a-zA-Z]/;

    return pattern.test(control.value) ? null : {
        'startsWithLetter': true
    };
}
```

Formularz w Szablonie

Formularz możemy też stworzyć bezpośrednio w szablonie:

```
<form #myForm="ngForm" (ngSubmit)="save(myForm)">
  <input name="comment" [(ngModel)]="item.comment"
       required minlength="20" #comment="ngModel" />

    <span *ngIf="comment.dirty && comment.hasError('required')">
      Pole jest wymagane!
    </span>

    <input type="submit" value="Zapisz">
</form>
```

Obiekt kontrolujący formularz możemy uzyskać korzystając z lokalnej referencji

```
<form #nazwa="ngForm" (ngSubmit)="mojaMetoda(nazwa)">
```

Pipes

Filtry

Transformacja danych z Pipe

- Pipe to filtr, który przekształca przekazane do niego dane :

```
<p> Total: {{ items.total | currency }} </p>
```

- Można go skonfigurować przekazując parametry:

```
<p> Total: {{ items.total | currency:'PLN':true }} </p>
```

- Wynik działania jednego pipe można przekazać do kolejnego

```
<p> Score: {{ player.score | number | replace:'0':'-' }} </p>
```

- JSON pipe

```
<pre>{{ obiekt | json }}</pre>
```

Wbudowane: DatePipe, UpperCasePipe, LowerCasePipe, CurrencyPipe, PercentPipe, JsonPipe

Parametryzowane Pipes

Pipe to klasa implementująca interfejs PipeTransform i opisana dekoratorem @Pipe()

```
import { Pipe, PipeTransform} from '@angular/core';

@Pipe({
    name: 'censor'
})
export class CensorPipe implements PipeTransform {
    transform(input: string, character ? : string): string {
        return input.replace(/./g, character || '*');
    }
}
```

Filtry - Użycie

Aby użyć custom pipe w komponencie, należy go zadeklarować

(podobnie jak robimy to z dyrektywami)

```
import { Component } from '@angular/core';
import { CensorPipe } from './censor.pipe';

@Component({
    selector: 'my-censor',
    template: '<span>{{ message | censor }}</span>',
})
export class Hello {
    message: string = 'My secret sentence';
}
```

Stan i asynchroniczność

w filtrach

Większość pipes jest bezstanowe.

Wykonują przekształcenia za pomocą czystych funkcji, bez efektów ubocznych.

Działają w szablonie jak “cache” dynamicznej wartości. Stateful pipes (np. AsyncPipe) zarządzają stanem przekazywanych danych

Definiując stateful pipe, oznaczamy ją jako pure: false

```
@Pipe({
  name: 'myStateful',
 pure: false
})
```

AsyncPipe przyjmuje jako wejście Promise lub Observable i przechowuje subskrypcję, żeby później zwrócić wartość

```
    <div *ngFor=”item in $stream | async”>{{item}}</div>
```

Routing

Angular Component Router

Konfiguracja Routingu

```
import { RouterModule }   from '@angular/router';


const routingModule = RouterModule.forRoot([
      { path: 'todos', component: TodosComponent },
      { path: '', component: HomeComponent },
      { path: '**', component: PageNotFoundComponent }
]);


@NgModule({
  imports: [
    BrowserModule,
    routingModule,
```

Router mapuje ścieżkę w URL na ustalony komponent. Kolejność podawania reguł ma znaczenie.

Komponent pojawi się w szablonie w miejscu wskazanym dyrektywą:

```
<router-outlet></router-outlet>
```

Routing dla Sub-Modułu

```
@NgModule({
  imports: [
    RouterModule.forChild([
      {
        path: 'heroes',
        component: HeroListComponent
      },
      {
        path: 'hero/:id',
        component: HeroDetailComponent
      }
    ])
  ],
  exports: [
    RouterModule
  ]
}) class SubModuleX{}
```

RouterModule.forRoot()

- tworzy moduł dla głównego modułu.

RouterModule.forChild()

- pozwala by submoduły miały własny częściowy routing.

RoutingModule tworzy cały moduł, by udostępnić wraz z nim potrzebne narzędzia - usługi i dyrektywy dla tego routingu!

Routing Lazy loading

```
const rotues: Routes = [
  { path: 'heroes',
    loadChilren: import('./heroes/heroes.module')
              .then(m => m.HeroesModule)
  },
]

@NgModule({
  imports: [ RouterModule.forRoot(routes) ],
  exports: [ RouterModule ]
})
```

Przekazywanie parametrów

```
{ path: 'todo/:id', component: TodoDetailComponent }
//  http://localhost/todo/15
```

Adresy możemy parametryzować - Wstrzykując usługi ActivatedRoute lub Params mamy dostęp do parametrów

```
import { Router, ActivatedRoute, Params } from '@angular/router';
constructor(
  private route: ActivatedRoute,
  private router: Router,
  private service: TodosService ) {}
// Usługa Router pozwala na programistyczną nawigację:
onSelect(todo: Todo) {
    this.router.navigate(['/todo', todo.id]);
```

Linkowanie do ścieżek routera

Dyrektywa RouterLink aktywuje daną ścieżkę routingu po kliknięciu w element. Ścieżka może składać się z wielu poziomów - podanych jako tablica:

```
<nav>
    <a routerLink="/todos" routerLinkActive="active">Todos App</a>

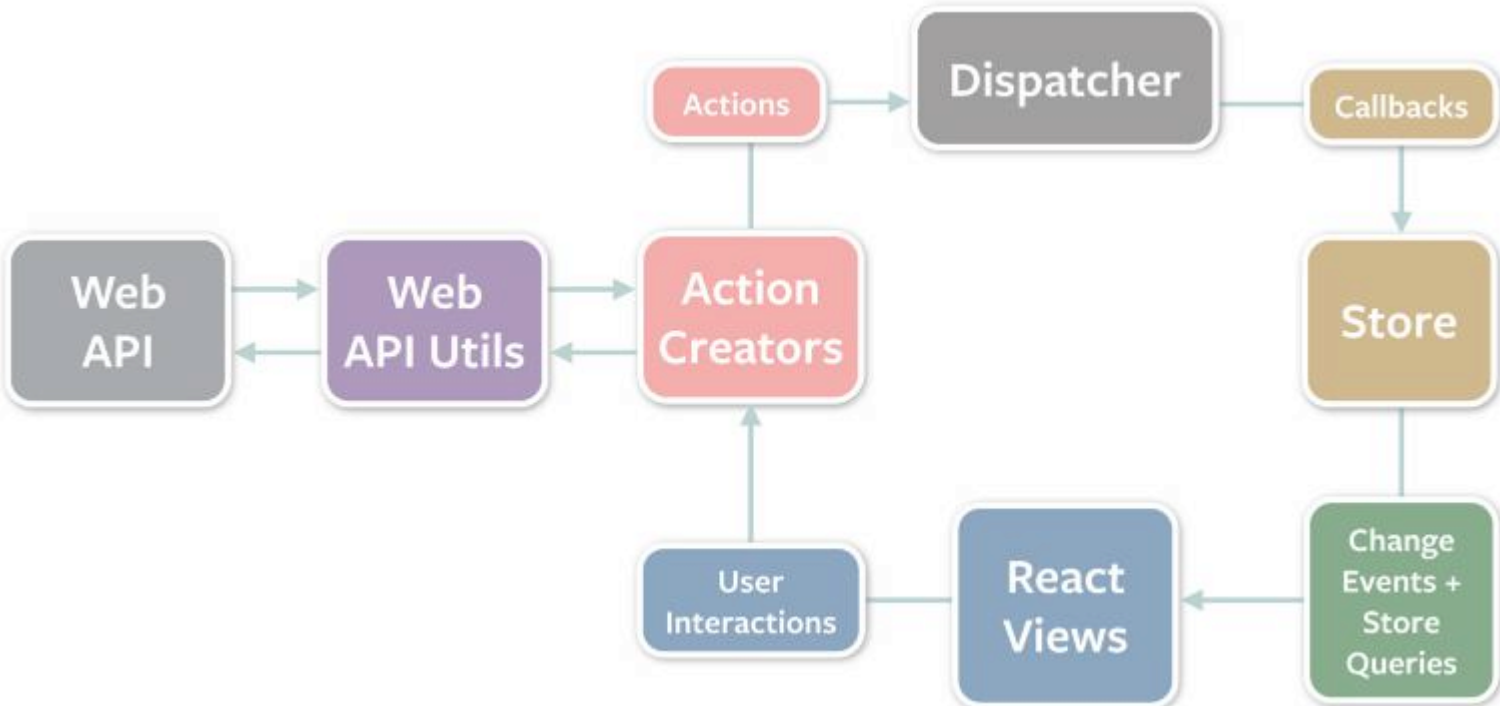
    <a [routerLink]="['/todo', todo.id ]" routerLinkActive="favourite-active">
My favourite Todo
</a>
</nav>
```

RouterLinkActive to dyrektywa która dodaje i usuwa podaną klasę CSS gdy ścieżka jest aktywna lub nie

NGRX

Reactive State for Angular

Wzorzec Flux/Redux



Actions

```
const ADD_TODO = {
  type:'ADD_TODO',
  payload: { todo: {  ...  }}
}
```

```
class AddTodo extends Action{
  readonly type = 'ADD_TODO'
  constructor(
    readonly payload: { todo: Todo }
  ){}
}
```

```
export const login = createAction("AddTodo",
props<{
  todo: Todo;
}>());
```

Reducer

```
export const counter = (state: number = 0, action: Action) => {
    switch(action.type){
        case 'INCREMENT':
            return state + 1;
        case 'DECREMENT':
            return state - 1;
        default:
            return state;
    }
};
```

Dispatcher

```
export interface Action {
  type: string;
  payload?: any;
}

// Proste działanie bez ładunku
dispatch({type: 'DECREMENT'});

// Działanie z powiązanym ładunkiem
dispatch({type: ADD_TODO, payload: {id: 1, message: 'Learn ngrx/store', completed: true}})
```

NGRX

Biblioteka do zarządzania złożonym stanem aplikacji w przewidywalny sposób.

- Normalizacja stanu i serializacja - Każda zmiana stanu jest wprowadzana w znormalizowany sposób.Serializacja pozwala na łatwe rejestrowanie zmian lub przechowywanie stanu, np.W localStorage
- Silne typowanie - ngrx is written in TypeScript and is strongly typed
- Encapsulacja - Widok, stan aplikacji i efekty są obsługiwane osobno
- Łatwość testowania - ngrx pozwala oddzielić stan aplikacji od widoku
- Efektywność - Ze względu na niemutowalny stan możliwe jest selektywne odświeżenie widoku z onPush

Nie potrzebujesz NGRX

Chyba że:

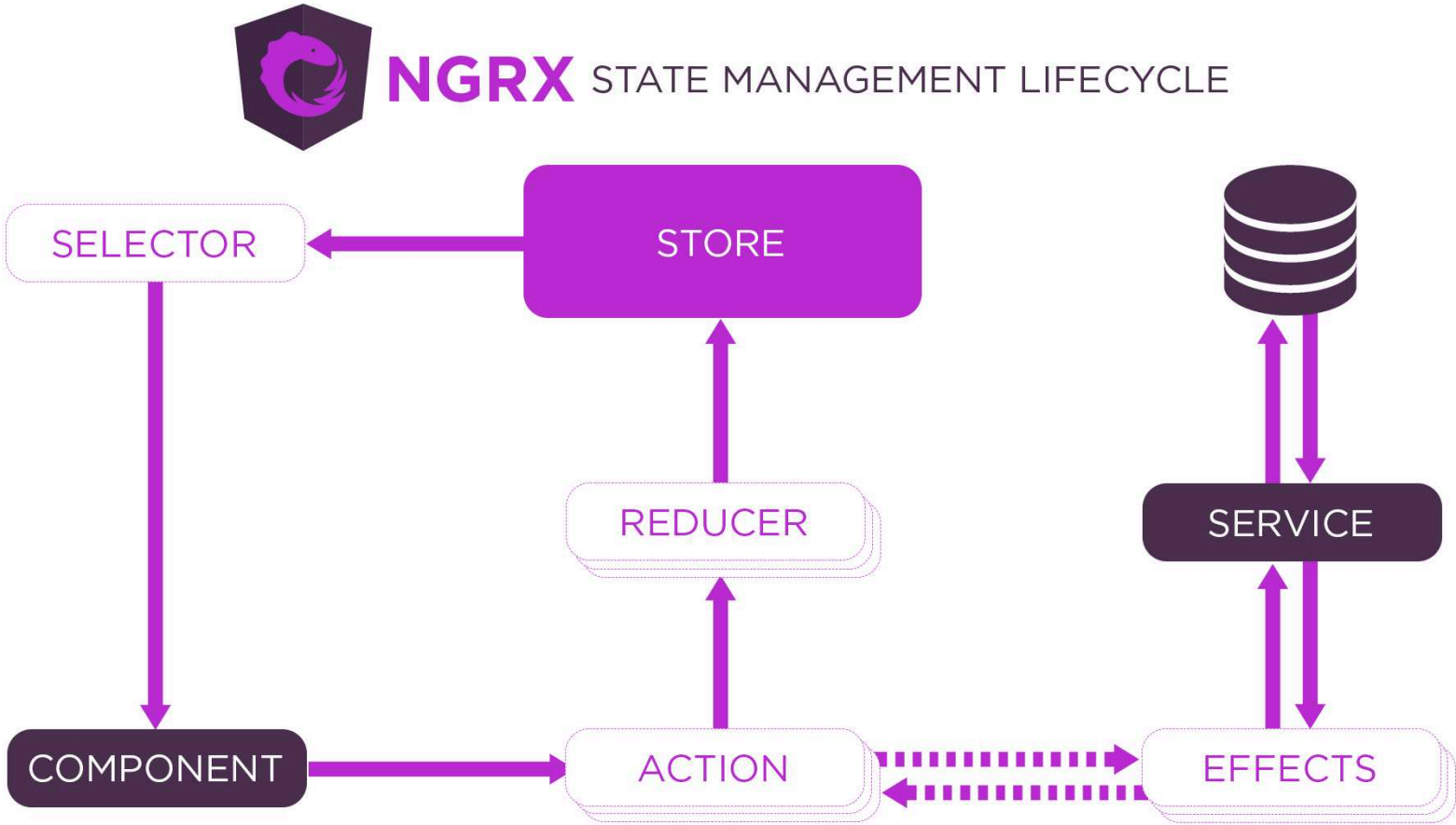
- Stan jest współdzielony przez wiele komponentów i usług
- Hydracja i hibernacja stanu aplikacji jest wymagana
- Łatwa Dostępność stanu pomiędzy ekranami jest wymagagana
- Aktualizacja współdzielonego stanu wymaga efektów ubocznych
- Stan aplikacji modyfikowany jest z wielu źródeł

MINUSY:

- NgRx dodaje kilka warstw złożoności do aplikacji. Jeśli aplikacja nie jest złożona NgRx nie jest potrzebny.
- Wymagana jest dobra znajomość RxJS i koncepcji Redux.

Elementy NGRX

- Store - magazyn danych
- Devtools - narzędzia deweloperskie
- Effects - warstwa obsługi efektów ubocznych
- Router Store - integracja z angular router
- Entity - menedżer kolekcji encji NgRx
- Data - narzędzia synchronizacji danych
- Schematics - generatory kodu dla NgRx


NGRX STATE MANAGEMENT LIFECYCLE
SELECTOR
STORE
REDUCER
SERVICE
COMPONENT
ACTION
EFFECTS

Instalacja NGRX

```
npm install --save @ngrx/store @ngrx/effects @ngrx/entity @ngrx/store-devtools
```

lub

```
npm install @ngrx/{store,store-devtools,entity,effects}
```

lub

```
ng add @ngrx/schematics@latest
ng add @ngrx/store@latest
```

Generatory NGRX

ng generate @ngrx/schematics:store --name=State --module=app --no-flat --root --stateInterface=AppState
ng generate @ngrx/schematics:effect --name=App --module=app --api --group --root ng generate
@ngrx/schematics:action --name=playlist --api --group ng generate @ngrx/schematics:reducer --
name=playlists --module=app --api --group --reducers=reducers/index.ts ng generate
@ngrx/schematics:selector --name=playlists --group

ng generate @ngrx/schematics:container --name=playlists/containers/playlists-ngrx --module=playlists --changeDetection=OnPush --stateInterface=AppState

Action Creators

```
import { createAction, props } from "@ngrx/store";

export const login = createAction( "[Login Page] Login", props<{
  username: string;
  password: string
}>());

export const homeScore = createAction("[Scoreboard Page] Home Score");
export const awayScore = createAction("[Scoreboard Page] Away Score");
```

Action Dispatcher

```
import {login} from ‘./action-creators.ts’

  onSubmit(username: string, password: string) {
    this.store.dispatch(login({
      username: username,
      password: password
    }));
  }
```

Reducer

```
const scoreboardReducer = createReducer( initialState,
  on(ScoreboardPageActions.homeScore, state ⇒ ({ ...state, home: state.home + 1 })),
  on(ScoreboardPageActions.awayScore, state ⇒ ({ ...state, away: state.away + 1 })),
  on(ScoreboardPageActions.resetScore, state ⇒ ({ home: 0, away: 0 })),
  on(ScoreboardPageActions.setScores, (state, { game }) ⇒ ({
    home: game.home,
    away: game.away
  })) );

export function reducer(state: State | undefined, action: Action) {
  return scoreboardReducer(state, action);
}
```

Rejestacja Modułu Reducera

```
import { NgModule } from "@angular/core";
import { StoreModule } from "@ngrx/store";
import * as fromScoreboard from "./reducers/scoreboard.reducer";

@NgModule({
  imports: [StoreModule.forRoot({ game: fromScoreboard.reducer })]
})
export class AppModule {}
```

Effects

Efekty oferują sposób na izolowanie i łatwe testowanie efektów ubocznych w aplikacji

```
export class PlaylistsEffects {
  constructor( playlistsService: PlaylistService,   private actions$: Actions) { }

  loadPlaylists = createEffect(() => this.actions$.pipe(
    // Match Actions
    ofType([PlaylistsActions.loadPlaylists, PlaylistsActions.refreshPlaylists]),
    switchMap(action => this.playlistsService.fetchCurrentUserPlaylists().pipe(
      // Call APIs
      mergeMap(resp => [
        // Dispatch Success
        PlaylistsActions.loadPlaylistsSuccess({ data: resp })
        // Dispatch additional actions
        PlaylistsActions.selectPlaylist({ id: resp[0].id || null } )
      ]),
      // Dispatch Errors
      catchError(err => [ PlaylistsActions.loadPlaylistsFailure({ error: err }) ])
    ))
  ))
}
```

Testy jednostkowe

poszczególnych elementów frameworka

Testowanie Pipes

i prostych usług

```
describe('TitleCasePipe', () => {
  // Pipe jest prostą bezstanową klasą
  // - nie ma tutaj specjalnej potrzeby inicjalizacji
  let pipe = new TitleCasePipe();

  it('transforms "abc" to "Abc"', () => {
    expect(pipe.transform('abc')).toBe('Abc');
  });
});
```

```
ng test
```

Testowanie komponentów

Komponent wymagać może inicjalizacji

np. dyrektyw z BrowserModule

```
beforeEach(() => {
  TestBed.configureTestingModule({
    declarations: [ BannerComponent ],
    imports: [ BrowserModule ]
  });

  // Tworzy Komponent i opakowauje w “Fixturę” ( ComponentFixture )
  fixture = TestBed.createComponent(BannerComponent);

  // Możemy dostać się bezpośrednio do klasy komponentu:
  comp = fixture.componentInstance;
});
```

Testowanie widoku

Jeżeli chcemy testować nie tylko klasę komponentu, ale także wygenerowany HTML, nie możemy zapomnieć o cyklu wykrywania zmian:

```
it('should display original title', () ⇒ {

  // Wywołaj ręcznie detekcje zmian by zakualizować widok (HTML):
  fixture.detectChanges();

  // Obiekt DebugElement posiada metody, np. do odnajdywania po CSS:
  de = fixture.debugElement.query(By.css('h1'));

  // Możemy porównać zawartość HTML z Obiektem komponentu:
  expect(de.nativeElement.textContent).toContain(comp.title);
});
```

Dziękuję za uwagę!

Pytania? ;-)